EXPERIMENT NO. 7

AIM: To identify functional dependencies in a denormalized Sales table and normalize it stepby-step from UNF to 3NF and BCNF. To verify lossless join and dependency preservation using SQL queries.

THEORY:

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion, and update anomalies. UNF, 1NF, 2NF, 3NF, and BCNF are different stages of normalization. Functional dependencies help identify relationships between attributes.

SQL QUERIES / PROGRAM:

Step 1: Create the Denormalized Sales Table

```
CREATE DATABASE SalesDB;

USE SalesDB;

CREATE TABLE Sales (
    OrderID INT,

    CustomerID INT,

    CustomerName VARCHAR(50),

    ProductID INT,

    ProductName VARCHAR(50),

    Quantity INT,

    Price DECIMAL(10,2),

    PRIMARY KEY (OrderID, ProductID)

);
```

Step 2: Insert Sample Data

```
INSERT INTO Sales VALUES

(101, 1, 'Amit', 201, 'Laptop', 1, 50000),

(101, 1, 'Amit', 202, 'Mouse', 2, 500),

(102, 2, 'Ravi', 201, 'Laptop', 1, 50000),

(103, 3, 'Neha', 203, 'Keyboard', 1, 1500),

(104, 2, 'Ravi', 202, 'Mouse', 3, 500);
```

Step 3: Identify Functional Dependencies

- OrderID → CustomerID
- CustomerID → CustomerName
- ProductID → ProductName, Price
- (OrderID, ProductID) → Quantity

The candidate key is (OrderID, ProductID).

Step 4: Convert UNF to 1NF

In 1NF, each attribute contains atomic values, and repeating groups are removed. The Sales table satisfies 1NF because each cell contains a single value.

SELECT * FROM Sales;

Step 5: Convert 1NF to 2NF

In 2NF, the table must be in 1NF and contain no partial functional dependencies.

Create separate tables for Orders, Customers, and Products.

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50)
);
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    FOREIGN KEY (CustomerID)
        REFERENCES Customers(CustomerID)
);
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Price DECIMAL(10,2)
);
```

```
CREATE TABLE OrderDetails (

    OrderID INT,

    ProductID INT,

    Quantity INT,

    PRIMARY KEY (OrderID, ProductID),

    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),

    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
```

Step 6: Convert 2NF to 3NF

In 3NF, the table must be in 2NF and contain no transitive dependencies. CustomerName is stored in Customers, so it depends directly on CustomerID rather than OrderID.

```
INSERT INTO Customers VALUES

(1, 'Amit'), (2, 'Ravi'), (3, 'Neha');

INSERT INTO Orders VALUES

(101, 1), (102, 2), (103, 3), (104, 2);

INSERT INTO Products VALUES

(201, 'Laptop', 50000),

(202, 'Mouse', 500),

(203, 'Keyboard', 1500);

INSERT INTO OrderDetails VALUES

(101, 201, 1), (101, 202, 2),

(102, 201, 1), (103, 203, 1),

(104, 202, 3);
```

Step 7: Convert 3NF to BCNF

A relation is in BCNF if every determinant of a non-trivial functional dependency is a superkey. The Customers, Orders, Products, and OrderDetails tables satisfy BCNF under the identified functional dependencies.

```
SELECT * FROM Customers;
```

```
SELECT * FROM Orders;

SELECT * FROM Products;

SELECT * FROM OrderDetails;
```

Step 8: Verify Lossless Join

A decomposition is lossless if joining the decomposed tables reconstructs the original information without generating spurious rows.

```
SELECT O.OrderID, C.CustomerID, C.CustomerName,

       P.ProductID, P.ProductName,

       OD.Quantity, P.Price

FROM Orders O

JOIN Customers C ON O.CustomerID = C.CustomerID

JOIN OrderDetails OD ON O.OrderID = OD.OrderID JOIN

Products P ON OD.ProductID = P.ProductID;
```

Step 9: Verify Dependency Preservation

Functional dependencies can be checked using the decomposed tables.

```
-- Verify CustomerID determines CustomerName

SELECT CustomerID, COUNT(DISTINCT CustomerName)

FROM Customers

GROUP BY CustomerID;

-- Verify ProductID determines ProductName and Price

SELECT ProductID, COUNT(DISTINCT ProductName),

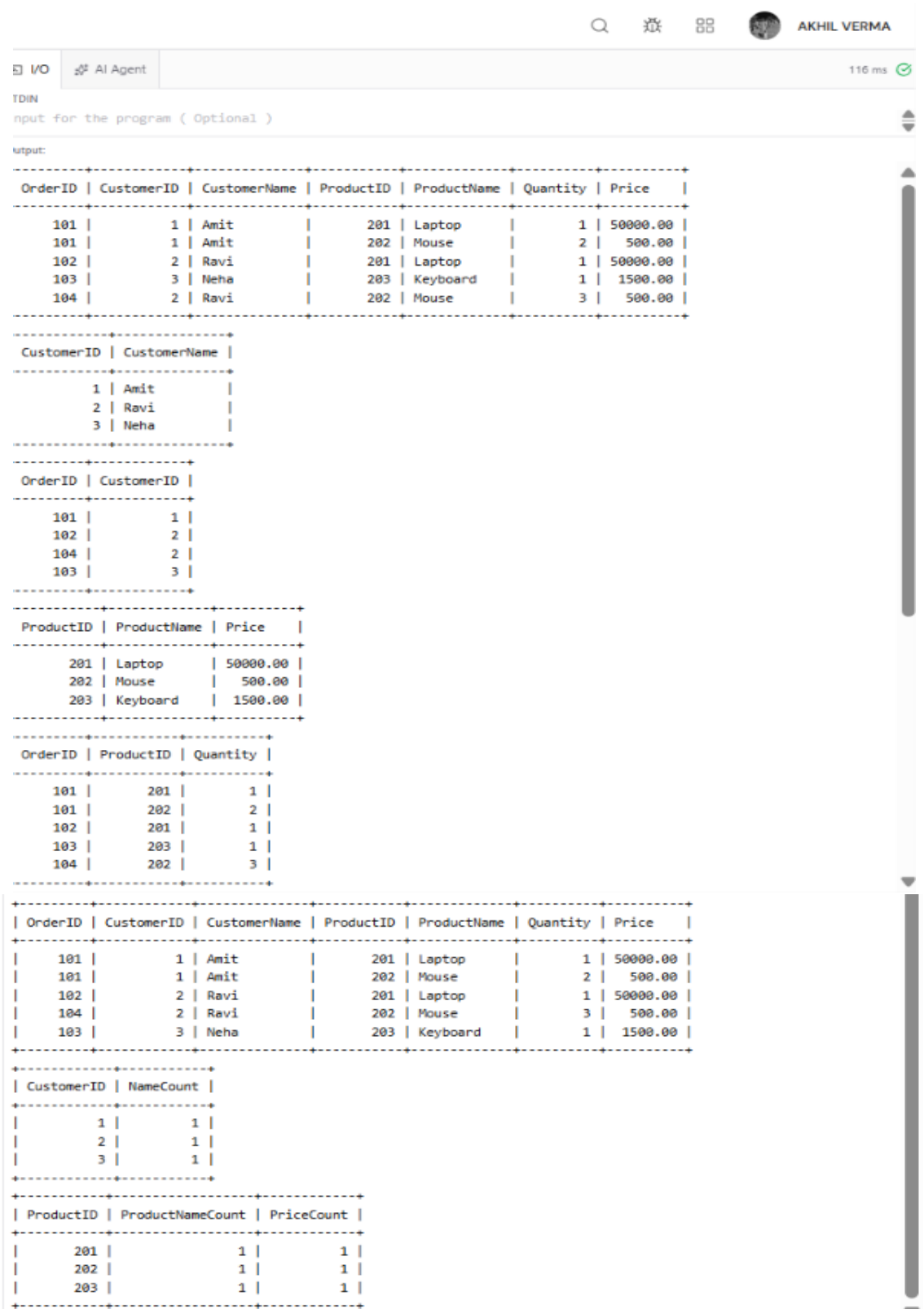
       COUNT(DISTINCT Price)

FROM Products

GROUP BY ProductID;
```

Each COUNT(DISTINCT ...) should return 1 for every valid determinant.

OUTPUT:



RESULT:

Thus, functional dependencies were identified, and the Sales table was normalized from UNF to 3NF and BCNF. Lossless join and dependency preservation were verified using SQL queries.